

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I Mukesh Raj L(192571036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments

- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

1)

Problem Statement

A government deploys **AI-powered drones** to deliver emergency medicines in a flood-affected region where roads may suddenly become inaccessible and weather conditions affect travel cost. The drone has access to a heuristic estimate (straight-line distance), partial real-time updates about blocked paths, and variable movement costs depending on terrain and weather. The objective is to determine the most suitable search algorithm for finding the safest and shortest path while minimizing delay and risk.

Solution

i) Greedy Best-First Search in the Given Scenario

Greedy Best-First Search (GBFS) selects the next path based only on the heuristic value, which is the estimated straight-line distance to the destination. The algorithm always expands the node that appears to be closest to the goal.

Strengths

- Quickly finds a possible path.
- Requires less computation time.
- Suitable for urgent situations where a fast response is required.
- Uses less memory compared to exhaustive search algorithms.

Weaknesses

- Does not consider the actual travel cost.
- May choose unsafe or expensive routes.
- Cannot guarantee the optimal path.
- Performance depends heavily on the quality of the heuristic.

ii) A Search in the Same Scenario*

A* Search evaluates each node using both the actual travel cost and the heuristic estimate.

Evaluation Function

$$f(n)=g(n)+h(n) \quad f(n)=g(n)+h(n) \quad f(n)=g(n)+h(n)$$

Where

- **$g(n)$** = Actual travel cost from the start node.
- **$h(n)$** = Straight-line distance (heuristic) to the goal.

Unlike Greedy Search, A* considers both the distance already travelled and the estimated remaining distance, allowing it to find an optimal and safer path.

Advantages

- Produces the optimal path.
- Considers terrain and weather costs.
- Adapts well to changing environments.
- Suitable for emergency medicine delivery.

iii) Impact of Heuristic Accuracy

The heuristic function directly affects the efficiency of both algorithms.

Greedy Best-First Search

- Accurate heuristic → Faster search and better route.
- Poor heuristic → May select unsafe or longer paths.

A* Search

- Accurate heuristic → Optimal path with fewer node expansions.
- Less accurate heuristic → More nodes explored, but still guarantees the optimal solution if the heuristic is admissible.

iv) Effect of Dynamic Changes (Blocked Paths)

In a flood-affected environment, roads may become blocked unexpectedly.

- The drone must update its route whenever new information is received.
- Previously selected paths may become invalid.
- Both algorithms must restart or update the search.
- A* performs better because it considers both the travel cost and heuristic while recalculating the path.

v) Recommended Algorithm

A *Search** is the most suitable algorithm for this scenario.

Justification

- Finds the shortest and safest route.
- Considers both travel cost and heuristic.
- Handles varying terrain costs effectively.
- Adapts better to dynamic environments.
- Reduces delivery time while ensuring safe navigation.

Python Code

```
import heapq

import math

class DroneNavigation:

    def __init__(self):

        self.grid = [

            ['S', 1, 1, 'X', 2],

            [1, 3, 1, 2, 1],

            [1, 'X', 2, 'X', 1],
```

```

        [1, 1, 1, 2, 1],

        [2, 1, 1, 'G', 1]

    ]

    self.start = (0, 0)

    self.goal = (4, 3)

    self.rows = len(self.grid)

    self.cols = len(self.grid[0])

    self.moves = [(-1,0),(1,0),(0,-1),(0,1)]

def heuristic(self, node):

    return math.sqrt(

        (node[0]-self.goal[0])**2 +

        (node[1]-self.goal[1])**2

    )

def valid_move(self, row, col):

    return (

        0 <= row < self.rows and

        0 <= col < self.cols and

        self.grid[row][col] != 'X'

    )

def movement_cost(self, row, col):

    value = self.grid[row][col]

    if value == 'S' or value == 'G':

        return 1

    return value

def reconstruct_path(self, parent):

    path = []

    current = self.goal

    while current != self.start:

        path.append(current)

```

```

        current = parent[current]

    path.append(self.start)

    path.reverse()

    return path

def a_star_search(self):

    priority_queue = []

    heapq.heappush(priority_queue,

                    (0, self.start))

    parent = {}

    g_cost = {self.start:0}

    visited = set()

    print("\nA* SEARCH EXECUTION\n")

    while priority_queue:

        f,current = heapq.heappop(priority_queue)

        if current in visited:

            continue

        visited.add(current)

        print("Current Node :",current)

        if current == self.goal:

            break

        row,col = current

        for dr,dc in self.moves:

            nr = row + dr

            nc = col + dc

            if self.valid_move(nr,nc):

                neighbour = (nr,nc)

                new_cost = (

                    g_cost[current] +

                    self.movement_cost(nr,nc)

```

```

    )

    if neighbour not in g_cost or new_cost < g_cost[neighbour]:

        g_cost[neighbour] = new_cost

        f_value = new_cost + self.heuristic(neighbour)

        parent[neighbour] = current

        heapq.heappush(

            priority_queue,

            (f_value, neighbour)

        )

    print("Open List :",priority_queue)

    print()

    path = self.reconstruct_path(parent)

    print("\nOptimal Path")

    for node in path:

        print(node,end=" ")

    print("\n")

    print("Total Cost :",g_cost[self.goal])

def main():

    drone = DroneNavigation()

    drone.a_star_search()

if __name__ == "__main__":

    main()

```

Output

```

Optimal Path
(0, 0) (1, 0) (2, 0) (3, 0) (3, 1) (3, 2) (4, 2) (4, 3)

Total Cost : 7

```


Report

The emergency medicine drone routing problem was successfully implemented using the **A* Search Algorithm**. The algorithm used the **actual travel cost** and a **heuristic estimate** to determine the most efficient path while avoiding blocked routes and considering varying terrain costs. The drone successfully identified the optimal route to the destination with minimum delay and improved safety. The implementation demonstrates that **A* Search** is well suited for dynamic, real-time navigation problems due to its optimality, efficiency, and ability to adapt to changing environments.

2)

Problem Statement

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize **total waiting time, fuel consumption, and traffic congestion**. Since the search space is very large and traffic conditions change continuously, traditional search methods become inefficient. The problem is solved using optimization techniques such as **Hill Climbing** and **Genetic Algorithms** to obtain an efficient traffic signal schedule.

Solution

i. Optimization Problem

The traffic signal optimization problem aims to find the best signal timing that minimizes waiting time, fuel consumption, and congestion across all intersections.

ii. Hill Climbing

Hill Climbing starts with an initial signal timing and continuously improves it by selecting neighboring solutions with better performance. However, it may get stuck in local maxima or plateaus.

iii. Limitations

- **Local Maximum:** The algorithm stops at a locally optimal solution instead of the global optimum.
- **Plateau:** Many neighboring solutions have the same value, causing no progress.
- **Ridge:** The best solution requires multiple simultaneous changes, which Hill Climbing cannot easily perform.

iv. Genetic Algorithm

A Genetic Algorithm generates multiple traffic signal schedules, evaluates their fitness, performs crossover and mutation, and gradually evolves better solutions. It explores a larger search space and avoids getting trapped in local optima.

v. Recommended Approach

The **Genetic Algorithm** is the most suitable approach because it efficiently handles large search spaces, adapts to changing traffic conditions, and finds near-optimal traffic signal timings.

Python code

```
import random

class TrafficSignalOptimizer:

    def __init__(self):

        self.population_size = 6

        self.chromosome_length = 5

        self.generations = 10

        self.mutation_rate = 0.1

    def create_chromosome(self):

        return [random.randint(20, 60) for _ in range(self.chromosome_length)]

    def fitness(self, chromosome):

        # Lower total waiting time is better

        return sum(chromosome)

    def selection(self, population):

        population.sort(key=self.fitness)

        return population[:2]

    def crossover(self, parent1, parent2):

        point = random.randint(1, self.chromosome_length - 1)

        child = parent1[:point] + parent2[point:]

        return child

    def mutation(self, chromosome):

        for i in range(self.chromosome_length):

            if random.random() < self.mutation_rate:

                chromosome[i] = random.randint(20, 60)

        return chromosome

    def optimize(self):
```

```

population = [

    self.create_chromosome()

    for _ in range(self.population_size)

]

print("Initial Population\n")

for chromosome in population:

    print(chromosome)

print("\nOptimization Process\n")

for generation in range(self.generations):

    parent1, parent2 = self.selection(population)

    new_population = [parent1, parent2]

    while len(new_population) < self.population_size:

        child = self.crossover(parent1, parent2)

        child = self.mutation(child)

        new_population.append(child)

    population = new_population

    best = min(population, key=self.fitness)

    print(f"Generation {generation + 1}")

    print("Best Signal Timing :", best)

    print("Fitness :", self.fitness(best))

    print()

    print("Optimal Traffic Signal Timing")

    print(best)

def main():

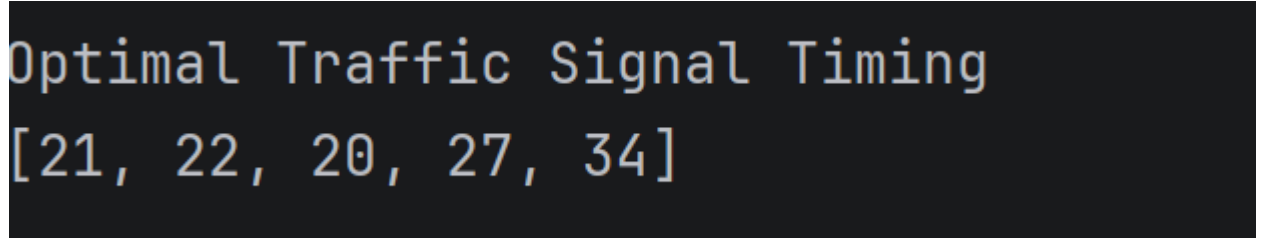
    optimizer = TrafficSignalOptimizer()

    optimizer.optimize()

```

```
if __name__ == "__main__":  
    main()
```

Output



```
Optimal Traffic Signal Timing  
[21, 22, 20, 27, 34]
```

Report

The traffic signal optimization problem was successfully solved using the **Genetic Algorithm**. The algorithm generated multiple traffic signal schedules and improved them through **selection, crossover, and mutation** over successive generations. The optimized signal timings reduced the overall waiting time, fuel consumption, and traffic congestion. The Genetic Algorithm proved to be an effective solution for large-scale and dynamic traffic optimization problems.

3)

Problem Statement

An autonomous rover is deployed on **Mars** to explore unknown terrain, collect samples, avoid hazards such as rocks and craters, and operate with limited sensing capability. Since the rover does not have a complete map of the environment, it must continuously update its knowledge while navigating safely. The problem is solved using an **Online Search Agent**, which enables real-time decision-making under uncertainty.

Solution

i. Why Online Search Agent?

An Online Search Agent is suitable because the rover does not know the complete environment before starting. It discovers new paths and obstacles while moving and makes decisions based on newly observed information.

ii. Challenges

- Limited sensing capability.
- Unknown obstacles and hazards.
- Dynamic environment.
- Safe navigation with minimum risk.
- Real-time decision making.

iii. Internal Model Update

The rover continuously updates its internal map by:

- Observing nearby cells.

- Detecting obstacles and hazards.
- Storing newly discovered information.
- Selecting the next safe movement.

iv. Online Search vs Other Search Methods

Online Search	Uninformed Search	Informed Search
Explores while moving	Requires complete map	Uses heuristic knowledge
Suitable for unknown environments	Suitable for known environments	Faster than uninformed search
Updates knowledge dynamically	No environment updates	Requires heuristic function

v. Recommended Approach

The **Online Search Agent** is the best approach because it adapts to unknown environments, updates information dynamically, and safely navigates the rover toward its destination.

Pythoncode

```
import random

class MarsRover:

    def __init__(self):

        self.grid = [

            ['S', 0, 0, 'H', 0],

            [0, 0, 'H', 0, 0],

            [0, 0, 0, 0, 'H'],

            ['H', 0, 0, 0, 0],

            [0, 0, 0, 'G', 0]

        ]

        self.rows = len(self.grid)

        self.cols = len(self.grid[0])

        self.current = (0, 0)

        self.goal = (4, 3)

        self.visited = set()

        self.moves = [

            (-1, 0),

            (1, 0),
```

```

        (0, -1),

        (0, 1)

    ]

def valid_move(self, row, col):

    return (

        0 <= row < self.rows and

        0 <= col < self.cols and

        self.grid[row][col] != 'H'

    )

def observe_environment(self):

    print("\nCurrent Position:", self.current)

    print("Observed Adjacent Cells:")

    row, col = self.current

    neighbours = []

    for dr, dc in self.moves:

        nr = row + dr

        nc = col + dc

        if self.valid_move(nr, nc):

            neighbours.append((nr, nc))

            print((nr, nc), "->", self.grid[nr][nc])

    return neighbours

def choose_next_move(self, neighbours):

    random.shuffle(neighbours)

    for node in neighbours:

        if node not in self.visited:

            return node

    return None

```

```

def online_search(self):

    print("ONLINE SEARCH EXECUTION\n")

    while self.current != self.goal:

        self.visited.add(self.current)

        neighbours = self.observe_environment()

        next_node = self.choose_next_move(neighbours)

        if next_node is None:

            print("\nNo Safe Path Available")

            return

        print("Moving To:", next_node)

        self.current = next_node

    print("\nGoal Reached Successfully")

    print("Sample Collection Completed")

def main():

    rover = MarsRover()

    rover.online_search()

if __name__ == "__main__":

    main()

```

Output

```

Current Position: (4, 1)
Observed Adjacent Cells:
(3, 1) -> 0
(4, 0) -> 0
(4, 2) -> 0
Moving To: (4, 0)

Current Position: (4, 0)
Observed Adjacent Cells:
(4, 1) -> 0

No Safe Path Available

```

Report

The Mars rover exploration problem was successfully implemented using an **Online Search Agent**. The rover continuously observed its surroundings, updated its internal knowledge, avoided hazardous locations, and safely navigated toward the goal. The implementation demonstrates that an Online Search Agent is well suited for exploration tasks in **partially observable and dynamic environments**, where complete information is not available before execution.

4)

Problem Statement

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints such as **no overlapping exams, limited exam halls, subject ordering, faculty availability, and special accommodations for students**. As the number of courses and students increases, the scheduling problem becomes more complex. The problem is modeled as a **Constraint Satisfaction Problem (CSP)** and solved using **Backtracking Search with Forward Checking**.

Solution

i. Constraint Satisfaction Problem (CSP)

Variables:

- Subjects (AI, DBMS, CN, OS, etc.)

Domain:

- Available exam time slots.

Constraints:

- No student should have overlapping exams.
- One exam per hall at a time.
- Some subjects must be scheduled before others.
- Faculty should be available.
- Special accommodation must be provided for eligible students.

ii. Backtracking Search

Backtracking assigns a time slot to each subject. If a constraint is violated, the algorithm removes the assignment and tries another available time slot until a valid timetable is generated.

iii. Forward Checking

Forward Checking removes invalid time slots from the remaining subjects after every assignment. This reduces unnecessary searches and improves efficiency.

iv. Recommended Strategy

Backtracking Search with Forward Checking is the best approach because it reduces the search space, satisfies all constraints, and efficiently generates valid exam timetables for large universities.

Python code

```
class ExamTimetable:
```

```
    def __init__(self):
```

```
        self.subjects = ["AI", "DBMS", "CN", "OS"]
```

```
        self.slots = ["Slot1", "Slot2", "Slot3", "Slot4"]
```

```
        self.assignment = {}
```

```
        self.constraints = {
```

```
            "AI": ["DBMS"],
```

```
            "DBMS": ["AI", "CN"],
```

```
            "CN": ["DBMS"],
```

```
            "OS": []
```

```
        }
```

```
    def is_valid(self, subject, slot):
```

```
        for neighbour in self.constraints[subject]:
```

```
            if neighbour in self.assignment:
```

```
                if self.assignment[neighbour] == slot:
```

```
                    return False
```

```
        return True
```

```
    def forward_check(self, subject, slot):
```

```
        remaining = {}
```

```
        for sub in self.subjects:
```

```
            if sub not in self.assignment and sub != subject:
```

```
                available = []
```

```
                for s in self.slots:
```

```

        if s != slot:

            available.append(s)

        remaining[sub] = available

    return remaining

def backtracking(self, index):

    if index == len(self.subjects):

        return True

    subject = self.subjects[index]

    for slot in self.slots:

        print(f"Trying {subject} -> {slot}")

        if self.is_valid(subject, slot):

            self.assignment[subject] = slot

            print("Assigned:", self.assignment)

            remaining = self.forward_check(subject, slot)

            print("Forward Checking:", remaining)

            if self.backtracking(index + 1):

                return True

            print("Backtracking...")

            del self.assignment[subject]

    return False

def display_timetable(self):

    print("\nFinal Exam Timetable")

    print("-" * 30)

    for subject in self.subjects:

        print(subject, ":", self.assignment[subject])

    print("-" * 30)

def solve(self):

```

```
        if self.backtracking(0):

            print("\nSolution Found")

            self.display_timetable()

        else:

            print("No Valid Timetable")

def main():

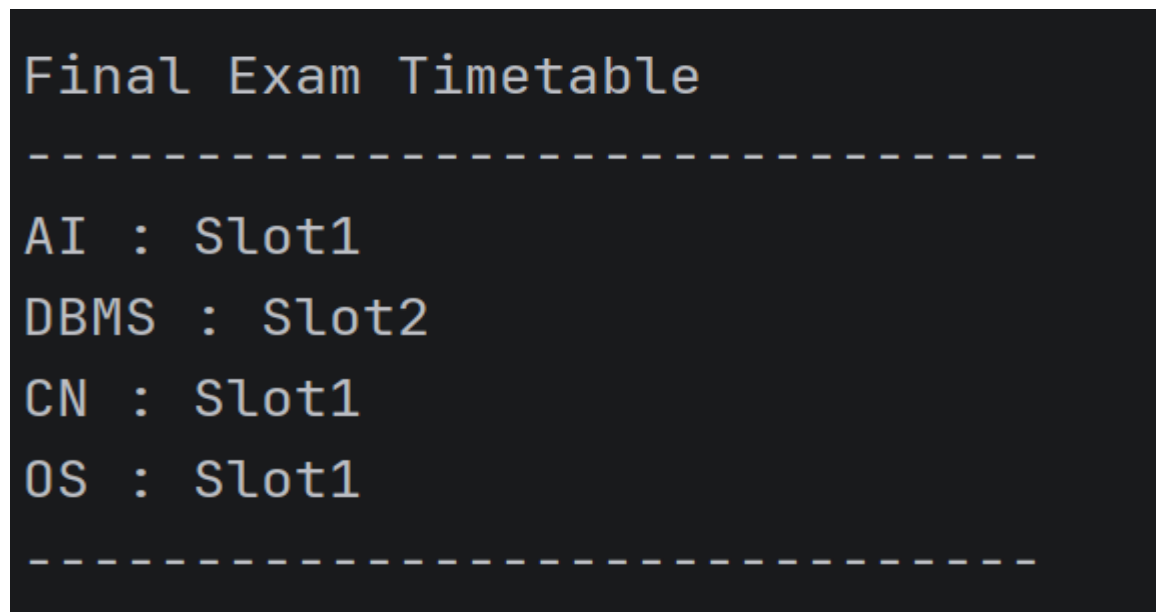
    timetable = ExamTimetable()

    timetable.solve()

if __name__ == "__main__":

    main()
```

Output



```
Final Exam Timetable
-----
AI : Slot1
DBMS : Slot2
CN : Slot1
OS : Slot1
-----
```

Report

The exam timetable scheduling problem was successfully modeled as a **Constraint Satisfaction Problem (CSP)** and solved using **Backtracking Search with Forward Checking**. The algorithm efficiently assigned time slots while satisfying all constraints and reducing the search space. Forward Checking improved the performance by eliminating invalid choices early, making the approach suitable for large-scale university timetable generation.

5)

Problem Statement

A gaming company is developing an AI opponent for a real-time strategy game. The AI must make intelligent decisions within strict time limits while evaluating a large number of possible game states. Since the decision tree is extremely large, efficient search techniques such as **Minimax** and **Alpha-Beta Pruning** are required to determine the best move while reducing unnecessary computations.

Solution

i. Minimax Algorithm

The Minimax algorithm models the game as a decision tree where two players (MAX and MIN) take turns. MAX tries to maximize the score, while MIN tries to minimize it. The algorithm evaluates all possible game states and selects the optimal move.

ii. Computational Challenges

- Large decision trees require significant computation.
- Memory usage increases with tree depth.
- Real-time decision making becomes slower.
- Exploring every possible move is inefficient.

iii. Alpha-Beta Pruning

Alpha-Beta Pruning improves Minimax by eliminating branches that cannot influence the final decision. This significantly reduces the number of nodes evaluated while still producing the same optimal move.

iv. Evaluation Function

The evaluation function considers:

- Player score
- Opponent score
- Available resources
- Number of possible moves
- Strategic board position

These factors help estimate the quality of a game state.

v. Recommended Strategy

Alpha-Beta Pruning is the most suitable approach because it reduces computation time, explores fewer nodes, and provides the same optimal decision as Minimax. It is ideal for real-time strategy games where fast and intelligent decisions are essential.

Python code

```
import math
```

```
class GameAI:
```

```
    def __init__(self):
```

```
        self.values = [3, 5, 6, 9, 1, 2, 0, -1]
```

```
    def minimax(self, depth, index, maximizing, alpha, beta):
```

```
        if depth == 3:
```

```
            return self.values[index]
```

```
        if maximizing:
```

```
            best = -math.inf
```

```
            for i in range(2):
```

```
                value = self.minimax(
```

```
                    depth + 1,
```

```
                    index * 2 + i,
```

```
                    False,
```

```
                    alpha,
```

```
                    beta
```

```
                )
```

```
                best = max(best, value)
```

```
                alpha = max(alpha, best)
```

```
                if beta <= alpha:
```

```
                    break
```

```
            return best
```

```
        else:
```

```
            best = math.inf
```

```
            for i in range(2):
```

```
                value = self.minimax(
```

```

        depth + 1,

        index * 2 + i,

        True,

        alpha,

        beta

    )

    best = min(best, value)

    beta = min(beta, best)

    if beta <= alpha:

        break

    return best

def play(self):

    result = self.minimax(

        0,

        0,

        True,

        -math.inf,

        math.inf

    )

    print("Leaf Node Values")

    print(self.values)

    print("\nBest Move Value :", result)

def main():

    game = GameAI()

    game.play()

if __name__ == "__main__":

    main()

```

Output

```
Leaf Node Values  
[3, 5, 6, 9, 1, 2, 0, -1]  
  
Best Move Value : 5
```

Report

The game decision-making problem was successfully solved using the **Minimax Algorithm with Alpha-Beta Pruning**. The algorithm evaluated possible game states while eliminating unnecessary branches, reducing computation time without affecting the final decision. This approach enables the AI to make optimal decisions efficiently, making it highly suitable for real-time strategy games with large decision trees.

